

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 12 SOLID PRINCIPLES



Disusun Oleh:
Yosafat Jacobus (105224016)

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN ILMU KOMPUTER
UNIVERSITAS PERTAMINA

2026

DAFTAR ISI

I. Pendahuluan.....	2
II. Variabel.....	3
III. Constructor dan Method.....	4
IV. Dokumentasi dan Pembahasan Code.....	6
V. Kesimpulan.....	19

I. Pendahuluan

SOLID Principles, yaitu lima prinsip desain Pemrograman Berorientasi Objek (OOP) yang diperkenalkan oleh Robert C. Martin. SOLID merupakan akronim dari Single Responsibility Principle (SRP), Open/Closed Principle (OCP), Liskov Substitution Principle (LSP), Interface Segregation Principle (ISP), dan Dependency Inversion Principle (DIP).

Kelima prinsip ini bertujuan untuk membuat kode perangkat lunak menjadi lebih mudah dipahami, fleksibel, terukur (scalable), dan mudah dirawat (maintainable). Pada Take Home Task ini, dilakukan analisis terhadap sistem SIAKAD yang melanggar prinsip-prinsip SOLID beserta perancangan ulang struktur kodenya agar menjadi tangkas, tidak mudah rusak, dan siap mengakomodasi perubahan di masa depan.

II. Variabel

No	Nama Variabel	Tipe Data	Fungsi
1	nama	String	Menyimpan nama mahasiswa
2	nim	String	Menyimpan NIM mahasiswa
3	jalurMasuk	String	Menyimpan jalur penerimaan mahasiswa
4	namaMK	String	Menyimpan nama mata kuliah
5	sks	int	Menyimpan jumlah SKS mata kuliah
6	prasyarat	List<String>	Menyimpan daftar prasyarat mata kuliah
7	daftarKRS	List<MataKuliah>	Menyimpan daftar mata kuliah dalam KRS
8	mahasiswa	Mahasiswa	Menyimpan objek mahasiswa pemilik KRS
9	validator	ValidatorPreSyarat	Objek untuk memvalidasi prasyarat MK
10	kalkulator	KalkulatorUKT	Objek untuk menghitung tagihan UKT
11	pencetak	PencetakKRS	Objek untuk mencetak draf KRS ke PDF
12	repository	KRSRepository	Objek untuk menyimpan data KRS ke database
13	tarifPerSKS	double	Tarif per SKS untuk perhitungan UKT

14	diskon	double	Persentase diskon UKT
----	--------	--------	-----------------------

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Fungsi
1	Mahasiswa (String , String, String)	Constructor	Menginisialisasi objek Mahasiswa dengan nama, NIM, dan jalur masuk
2	getNama ()	Getter	Mengembalikan nama mahasiswa
3	getNim ()	Getter	Mengembalikan NIM mahasiswa
4	getJalurMasuk ()	Getter	Mengembalikan jalur masuk mahasiswa
5	getNamaMK ()	Getter	Mengembalikan nama mata kuliah
6	getSks ()	Getter	Mengembalikan jumlah SKS
7	getPreSyarat ()	Getter	Mengembalikan daftar prasyarat MK
8	alokasiAsistenLa b ()	Method	Mengalokasikan asisten lab untuk praktikum
9	cekPeralatanPrak tikum ()	Method	Mengecek peralatan yang dibutuhkan untuk praktikum

10	validasi (Mahasiswa, MataKuliah)	Method	Memvalidasi prasyarat MK untuk mahasiswa
11	hitungUKT (Mahasiswa, List<MataKuliah>)	Method	Menghitung tagihan UKT mahasiswa
12	cetakKRS (Mahasiswa, List<MataKuliah>)	Method	Mencetak draf KRS ke bentuk file PDF
13	simpan (KRS)	Method	Menyimpan data KRS ke database
14	prosesKRS (Mahasiswa, List<MataKuliah>)	Method	Mengorkestrasi seluruh proses pengisian KRS

IV. Dokumentasi dan Pembahasan Code

Model Data:

Mahasiswa.java

```
// Kelas model data Mahasiswa untuk bertanggung jawab menyimpan data mahasiswa (SRP)
public class Mahasiswa {
    private String nama;
    private String nim;
    private String jalurMasuk;

    public Mahasiswa(String nama, String nim, String jalurMasuk) {
        this.nama = nama;
        this.nim = nim;
        this.jalurMasuk = jalurMasuk;
    }

    public String getNama() { return nama; }
    public String getNim() { return nim; }
    public String getJalurMasuk() { return jalurMasuk; }
}
```

Kelas model data yang merepresentasikan entitas Mahasiswa dengan atribut nama, NIM, dan jalur masuk. Menerapkan Single Responsibility Principle (SRP), kelas ini hanya bertanggung jawab menyimpan dan menyediakan data mahasiswa, tidak mengurus logika bisnis lainnya.

MataKuliah.java

```
import java.util.List;

// Interface dasar untuk semua jenis mata kuliah hanya berisi operasi umum (ISP)
public interface MataKuliah {
    String getNamaMK();
    int getSks();
    List<String> getPreSyarat();
}
```

Interface dasar yang mendefinisikan operasi umum untuk semua jenis mata kuliah. Menerapkan Interface Segregation Principle (ISP). Interface ini hanya

berisi metode yang relevan untuk semua jenis MK (nama, SKS, prasyarat), tanpa memaksakan metode khusus lab seperti alokasiAsistenLab().

Praktikumable.java

```
// Interface tambahan khusus MK yang memiliki kegiatan lab (ISP)
// Dipisahkan agar kelas Teori dan KKN tidak dipaksa
mengimplementasikannya
public interface Praktikumable {
    void alokasiAsistenLab();
    void cekPeralatanPraktikum();
}
```

Interface yang memisahkan operasi khusus praktikum dari interface MataKuliah. Menerapkan Interface Segregation Principle (ISP). interface ini hanya diimplementasikan oleh MataKuliahPraktikum, sehingga kelas MataKuliahTeori dan MataKuliahKKN tidak dipaksa mengimplementasikan metode yang tidak mereka butuhkan.

MataKuliahTeori.java

```
import java.util.List;

// MK Teori, hanya mengimplementasikan MataKuliah, tanpa metode lab
(LSP & ISP terpenuhi)
public class MataKuliahTeori implements MataKuliah {
    private String namaMK;
    private int sks;
    private List<String> prasyarat;

    public MataKuliahTeori(String namaMK, int sks, List<String>
prasyarat) {
        this.namaMK = namaMK;
        this.sks = sks;
        this.prasyarat = prasyarat;
    }
}
```

```

public String getNamaMK() { return namaMK; }
public int getSks() { return sks; }
public List<String> getPreSyarat() { return prasyarat; }
}

```

Kelas mata kuliah Teori yang hanya mengimplementasikan interface MataKuliah. Menerapkan LSP dan ISP sama seperti MataKuliahKKN, kelas ini tidak dipaksa mengimplementasikan metode lab sehingga tidak merusak kontrak dari kelas induk saat diiterasi.

MataKuliahPraktikum.java

```

import java.util.List;

// MK Praktikum untuk mengimplementasikan MataKuliah dan Praktikumable
// (ISP terpenuhi)
public class MataKuliahPraktikum implements MataKuliah, Praktikumable {
    private String namaMK;
    private int sks;
    private List<String> prasyarat;

    public MataKuliahPraktikum(String namaMK, int sks, List<String>
prasyarat) {
        this.namaMK = namaMK;
        this.sks = sks;
        this.prasyarat = prasyarat;
    }

    public String getNamaMK() { return namaMK; }
    public int getSks() { return sks; }
    public List<String> getPreSyarat() { return prasyarat; }

    // Implementasi nyata karena Praktikum memang membutuhkan asisten
lab
    public void alokasiAsistenLab() {
        System.out.println("Mengalokasikan asisten lab untuk " +
namaMK);
    }

    // Implementasi nyata karena Praktikum memang membutuhkan peralatan
    public void cekPeralatanPraktikum() {

```

```

        System.out.println("Mengecek peralatan praktikum untuk " +
namaMK);
    }
}

```

Kelas mata kuliah Praktikum yang mengimplementasikan dua interface: MataKuliah dan Praktikumable. Menerapkan ISP, hanya kelas ini yang mengimplementasikan metode lab karena memang membutuhkannya. Metode alokasiAsistenLab() dan cekPeralatanPraktikum() memiliki implementasi nyata, bukan lemparan exception.

MataKuliahKKN.java

```

import java.util.List;

// MK KKN/Kuliah Lapangan untuk mengimplementasikan MataKuliah, tanpa
metode lab (LSP & ISP)
public class MataKuliahKKN implements MataKuliah {
    private String namaMK;
    private int sks;
    private List<String> prasyarat;

    public MataKuliahKKN(String namaMK, int sks, List<String> prasyarat)
    {
        this.namaMK = namaMK;
        this.sks = sks;
        this.prasyarat = prasyarat;
    }

    public String getNamaMK() { return namaMK; }
    public int getSks() { return sks; }
    public List<String> getPreSyarat() { return prasyarat; }
}

```

Kelas mata kuliah KKN/Kuliah Lapangan yang hanya mengimplementasikan interface MataKuliah. Menerapkan LSP dan ISP, kelas ini tidak dipaksa mengimplementasikan metode lab yang tidak relevan, sehingga tidak ada lemparan UnsupportedOperationException dan dapat menggantikan tipe MataKuliah dengan aman.

KRS.java

```
import java.util.List;

// Kelas model data KRS untuk menyimpan data KRS mahasiswa
public class KRS {
    private Mahasiswa mahasiswa;
    private List<MataKuliah> daftarKRS;

    public KRS(Mahasiswa mahasiswa, List<MataKuliah> daftarKRS) {
        this.mahasiswa = mahasiswa;
        this.daftarKRS = daftarKRS;
    }

    public Mahasiswa getMahasiswa() { return mahasiswa; }
    public List<MataKuliah> getDaftarKRS() { return daftarKRS; }
}
```

Kelas model data yang merepresentasikan Kartu Rencana Studi. Menyimpan objek Mahasiswa dan daftar MataKuliah yang diambil. Menerapkan Single Responsibility Principle (SRP) kelas ini hanya bertanggung jawab menyimpan data KRS, tidak mengurus validasi, perhitungan, maupun penyimpanan ke database.

Service Layer (pengolah):

ValidatorPreSyarat.java

```
// Kelas terpisah yang hanya bertanggung jawab memvalidasi prasyarat MK (SRP)
public class ValidatorPreSyarat {
    public boolean validasi(Mahasiswa mhs, MataKuliah mk) {
        System.out.println("Memvalidasi prasyarat " + mk.getNamaMK() + " untuk " + mhs.getNama());
        // Logika validasi prasyarat
        return true;
    }
}
```

Kelas yang hanya bertanggung jawab memvalidasi prasyarat mata kuliah mahasiswa. Menerapkan Single Responsibility Principle (SRP), logika validasi dipisahkan dari kelas raksasa SistemKRSManager, sehingga perubahan aturan validasi tidak mempengaruhi komponen lain seperti pencetakan PDF atau penyimpanan database.

KalkulatorUKT.java

```
import java.util.List;

// Interface strategi perhitungan UKT untuk memungkinkan penambahan
jalur baru tanpa modifikasi (OCP)
public interface KalkulatorUKT {
    double hitungUKT(Mahasiswa mhs, List<MataKuliah> daftarMK);
}
```

Interface strategi untuk perhitungan Uang Kuliah Tunggal (UKT). Menerapkan Open/Closed Principle (OCP) — setiap jalur masuk mahasiswa memiliki kelas strateginya sendiri yang mengimplementasikan interface ini. Penambahan jalur baru cukup membuat kelas baru, tanpa memodifikasi kode yang sudah ada.

KalkulatorUKTReguler.java

```
import java.util.List;

// Perhitungan UKT untuk mahasiswa Reguler
public class KalkulatorUKTReguler implements KalkulatorUKT {
    private double tarifPerSKS = 500000;

    public double hitungUKT(Mahasiswa mhs, List<MataKuliah> daftarMK) {
        int totalSKS =
daftarMK.stream().mapToInt(MataKuliah::getSks).sum();
        double total = totalSKS * tarifPerSKS;
        System.out.println("UKT Reguler untuk " + mhs.getNama() + ": Rp"
+ total);
        return total;
    }
}
```

Implementasi strategi perhitungan UKT untuk mahasiswa jalur Reguler dengan tarif Rp500.000 per SKS. Menerapkan OCP dan menggantikan blok if-else yang bertumpuk di kelas lama SistemKRSManager.

KalkulatorUKTKaryawan.java

```
import java.util.List;

// Perhitungan UKT untuk mahasiswa Karyawan
public class KalkulatorUKTKaryawan implements KalkulatorUKT {
    private double tarifPerSKS = 600000;

    public double hitungUKT(Mahasiswa mhs, List<MataKuliah> daftarMK) {
        int totalSKS =
daftarMK.stream().mapToInt(MataKuliah::getSks).sum();
        double total = totalSKS * tarifPerSKS;
        System.out.println("UKT Karyawan untuk " + mhs.getNama() + ":
Rp" + total);
        return total;
    }
}
```

Implementasi strategi perhitungan UKT untuk mahasiswa jalur Karyawan dengan tarif Rp600.000 per SKS. Menerapkan OCP, pola yang sama dengan strategi lainnya, menunjukkan bahwa setiap jalur terisolasi dalam kelasnya sendiri.

KalkulatorUKTInternasional.java

```
import java.util.List;

// Perhitungan UKT untuk mahasiswa Internasional
public class KalkulatorUKTInternasional implements KalkulatorUKT {
    private double tarifPerSKS = 1000000;

    public double hitungUKT(Mahasiswa mhs, List<MataKuliah> daftarMK) {
        int totalSKS =
daftarMK.stream().mapToInt(MataKuliah::getSks).sum();
        double total = totalSKS * tarifPerSKS;
        System.out.println("UKT Internasional untuk " + mhs.getNama() +
": Rp" + total);
        return total;
    }
}
```

```
}
```

Implementasi strategi perhitungan UKT untuk mahasiswa jalur Internasional dengan tarif Rp1.000.000 per SKS. Menerapkan OCP dan menghitung total SKS dari daftar mata kuliah lalu dikalikan tarif, tanpa mempengaruhi strategi jalur lain.

KalkulatorUKTBidikmisi.java

```
import java.util.List;

// Perhitungan UKT untuk mahasiswa Bidikmisi ada diskon penuh (OCP)
public class KalkulatorUKTBidikmisi implements KalkulatorUKT {

    public double hitungUKT(Mahasiswa mhs, List<MataKuliah> daftarMK) {
        double total = 0;
        System.out.println("UKT Bidikmisi untuk " + mhs.getNama() + ":
Rp" + total + " (ditanggung negara)");
        return total;
    }
}
```

Implementasi strategi perhitungan UKT untuk mahasiswa jalur Bidikmisi. Total UKT adalah Rp0 karena ditanggung negara. Menerapkan OCP, kelas ini berdiri sendiri dan tidak mempengaruhi perhitungan UKT jalur lainnya.

KalkulatorUKTMBKM.java

```
import java.util.List;

// Perhitungan UKT untuk program MBKM agar siap ditambahkan tanpa
modifikasi kelas lain (OCP)
public class KalkulatorUKTMBKM implements KalkulatorUKT {
    private double tarifPerSKS = 350000;

    public double hitungUKT(Mahasiswa mhs, List<MataKuliah> daftarMK) {
        int totalSKS =
daftarMK.stream().mapToInt(MataKuliah::getSks).sum();
```

```

        double total = totalSKS * tarifPerSKS;
        System.out.println("UKT MBKM untuk " + mhs.getNama() + ": Rp" +
total);
        return total;
    }
}

```

Implementasi strategi perhitungan UKT untuk program Merdeka Belajar Kampus Merdeka (MBKM) dengan tarif Rp350.000 per SKS. Kelas ini membuktikan OCP. Jalur MBKM yang merupakan kebutuhan baru dari rektorat dapat ditambahkan hanya dengan membuat kelas baru tanpa menyentuh kode yang sudah ada.

PencetakKRS.java

```

import java.util.List;

// Kelas terpisah yang hanya bertanggung jawab mencetak KRS ke format
PDF (SRP)
public class PencetakKRS {
    public void cetakKRS(Mahasiswa mhs, List<MataKuliah> daftarMK) {
        System.out.println("=== DRAF KRS ===");
        System.out.println("Nama: " + mhs.getNama());
        System.out.println("NIM: " + mhs.getNim());
        for (MataKuliah mk : daftarMK) {
            System.out.println("- " + mk.getNamaMK() + " (" +
mk.getSks() + " SKS)");
        }
        System.out.println("Mencetak ke format PDF...");
    }
}

```

Kelas yang hanya bertanggung jawab mencetak draf KRS ke format PDF. Menerapkan Single Responsibility Principle (SRP) . Logika pencetakan dipisahkan dari kelas raksasa SistemKRSManager, sehingga perubahan format kop surat tidak lagi merusak logika penyimpanan database.

Repository Layer (penyimpanan):

KRSRepository.java

```
// Interface abstraksi penyimpanan data KRS untuk modul tingkat tinggi
bergantung pada ini (DIP)
public interface KRSRepository {
    void simpan(KRS krs);
}
```

Interface abstraksi untuk penyimpanan data KRS ke database. Menerapkan Dependency Inversion Principle (DIP), modul tingkat tinggi (SistemKRS) bergantung pada interface ini, bukan pada implementasi konkret seperti MySQL atau Cloud NoSQL. Dengan begitu, migrasi database cukup membuat implementasi baru tanpa mengubah logika bisnis.

MySQLKRSRepository.java

```
// Implementasi penyimpanan MySQL sebagai detail tingkat rendah (DIP)
public class MySQLKRSRepository implements KRSRepository {
    public void simpan(KRS krs) {
        System.out.println("Menyimpan KRS " +
krs.getMahasiswa().getNama() + " ke MySQL Database...");
    }
}
```

Kelas implementasi konkret dari interface KRSRepository untuk penyimpanan ke MySQL. Menerapkan DIP, kelas ini adalah detail tingkat rendah yang bisa ditukar dengan CloudNoSQLKRSRepository tanpa mengubah kode di SistemKRS, karena SistemKRS bergantung pada abstraksi KRSRepository.

CloudNoSQLKRSRepository.java

```
// Implementasi penyimpanan Cloud NoSQL – bisa ditukar tanpa ubah kode
tingkat tinggi (DIP)
public class CloudNoSQLKRSRepository implements KRSRepository {
    public void simpan(KRS krs) {
        System.out.println("Menyimpan KRS " +
krs.getMahasiswa().getNama() + " ke Cloud NoSQL...");
    }
}
```

Kelas ini adalah implementasi konkret dari interface KRSRepository untuk penyimpanan ke Cloud NoSQL. Menerapkan Dependency Inversion Principle (DIP), kelas tingkat tinggi (SistemKRS) tidak perlu tahu detail database apa yang dipakai, cukup bergantung pada abstraksi KRSRepository. Kelas ini bisa langsung menggantikan MySQLKRSRepository tanpa mengubah satu baris pun kode di SistemKRS.

Orkestrasi (merakit semua)

SistemKRS.java

```
import java.util.List;

// Kelas orkestrasi yang mengoordinasi seluruh proses KRS
// Setiap tanggung jawab didelegasikan ke kelas terpisah (SRP)
// Bergantung pada abstraksi, bukan implementasi konkret (DIP)
public class SistemKRS {
    private ValidatorPreSyarat validator;
    private KalkulatorUKT kalkulator;
    private PencetakKRS pencetak;
    private KRSRepository repository;

    // Dependency Injection melalui constructor untuk memudahkan
    penukaran implementasi (DIP)
    public SistemKRS(ValidatorPreSyarat validator, KalkulatorUKT
kalkulator,
                    PencetakKRS pencetak, KRSRepository repository) {
        this.validator = validator;
        this.kalkulator = kalkulator;
    }
}
```

```

        this.pencetak = pencetak;
        this.repository = repository;
    }

    public void prosesKRS(Mahasiswa mhs, List<MataKuliah> daftarMK) {
        // Validasi prasyarat setiap MK
        for (MataKuliah mk : daftarMK) {
            validator.validasi(mhs, mk);
        }
        // Hitung tagihan UKT
        kalkulator.hitungUKT(mhs, daftarMK);
        // Cetak draf KRS
        pencetak.cetakKRS(mhs, daftarMK);
        // Simpan ke database
        KRS krs = new KRS(mhs, daftarMK);
        repository.simpan(krs);
        System.out.println("Proses KRS selesai untuk " + mhs.getNama());
    }
}

```

Kelas orkestrasi (facade) yang mengoordinasi seluruh proses pengisian KRS. Menerapkan SRP, kelas ini hanya mengoordinasi alur, setiap tanggung jawab didelegasikan ke kelas terpisah. Menerapkan DIP, dependency diinjeksikan melalui constructor, sehingga implementasi database, kalkulator UKT, dan komponen lainnya dapat ditukar secara fleksibel tanpa mengubah kode kelas ini.

Entry Point (demo):

Main.java

```

import java.util.Arrays;
import java.util.List;

// Kelas utama untuk mendemonstrasikan sistem SIAKAD yang telah
// dirancang ulang dengan SOLID
public class Main {
    public static void main(String[] args) {
        // Membuat objek mahasiswa
        Mahasiswa mhs = new Mahasiswa("Yosafat Jacobus", "105224016",
        "Reguler");
    }
}

```

```

        // Membuat daftar mata kuliah dengan berbagai jenis (LSP & ISP
        terpenuhi)
        List<MataKuliah> daftarMK = Arrays.asList(
            new MataKuliahTeori("Pemrograman Berorientasi Objek", 3,
Arrays.asList("Dasar Pemrograman")),
            new MataKuliahPraktikum("Praktikum PBO", 1,
Arrays.asList("PBO")),
            new MataKuliahKKN("Kuliah Kerja Nyata", 4, Arrays.asList())
        );

        // Skenario 1 menggunakan MySQL (sebelum migrasi)
        System.out.println("\n=== SKENARIO 1: MySQL + Mahasiswa Reguler
===");

        SistemKRS sistemMySQL = new SistemKRS(
            new ValidatorPreSyarat(),
            new KalkulatorUKTReguler(),        // Strategi UKT Reguler
(OCP)
            new PencetakKRS(),
            new MySQLKRSRepository()           // Implementasi MySQL (DIP)
        );
        sistemMySQL.prosesKRS(mhs, daftarMK);

        System.out.println();

        // Skenario 2 Migrasi ke Cloud NoSQL (tanpa ubah kode SistemKRS)
        System.out.println("=== SKENARIO 2: Cloud NoSQL + Mahasiswa
Bidikmisi ===");
        Mahasiswa mhsBidikmisi = new Mahasiswa("Pradana Akbar Razan",
"105224043", "Bidikmisi");
        SistemKRS sistemCloud = new SistemKRS(
            new ValidatorPreSyarat(),
            new KalkulatorUKTBidikmisi(),      // Strategi UKT Bidikmisi
(OCP)
            new PencetakKRS(),
            new CloudNoSQLKRSRepository()      // Implementasi Cloud
NoSQL (DIP)
        );
        sistemCloud.prosesKRS(mhsBidikmisi, daftarMK);

        System.out.println();

        // Skenario 3: Program MBKM (penambahan jalur baru tanpa
modifikasi)

```

```

        System.out.println("=== SKENARIO 3: Cloud NoSQL + Mahasiswa MBKM
        ===");
        Mahasiswa mhsMBKM = new Mahasiswa("Dwi Setiawan", "105224015",
        "MBKM");
        SistemKRS sistemMBKM = new SistemKRS(
            new ValidatorPreSyarat(),
            new KalkulatorUKTMBKM(),           // Jalur baru tanpa
        modifikasi kelas lama (OCP)
            new PencetakKRS(),
            new CloudNoSQLKRSRepository()
        );
        sistemMBKM.prosesKRS(mhsMBKM, daftarMK);
    }
}

```

Kelas utama yang mendemonstrasikan tiga skenario penggunaan sistem SIAKAD yang telah dirancang ulang. Skenario 1 menunjukkan penggunaan MySQL dengan mahasiswa Reguler, Skenario 2 menunjukkan migrasi ke Cloud NoSQL dengan mahasiswa Bidikmisi (DIP), dan Skenario 3 menunjukkan penambahan jalur MBKM tanpa modifikasi kode lama (OCP). Ketiga skenario menggunakan SistemKRS yang sama hanya implementasi yang diinjeksikan berbeda.

V. Kesimpulan

Pada Take Home Task Modul 12, dilakukan analisis dan perancangan ulang sistem SIAKAD yang awalnya melanggar kelima prinsip SOLID. Kelas raksasa SistemKRSManager yang memiliki banyak tanggung jawab dipecah menjadi kelas-kelas terpisah sesuai prinsip SRP (ValidatorPreSyarat, KalkulatorUKT, PencetakKRS, KRSRepository).

Perhitungan UKT yang semula menggunakan if-else bertumpuk dirancang ulang menggunakan pola Strategy dengan interface KalkulatorUKT sesuai prinsip OCP, sehingga jalur mahasiswa baru seperti MBKM dapat ditambahkan tanpa memodifikasi kode yang sudah ada. Interface OperasiMataKuliah yang terlalu gemuk dipecah menjadi MataKuliah dan Praktikumable agar kelas Teori dan KKN tidak dipaksa mengimplementasikan metode lab sesuai prinsip ISP dan LSP. Terakhir, ketergantungan langsung pada MySQLDatabaseConnection diganti dengan abstraksi KRSRepository melalui Dependency Injection sesuai prinsip DIP, sehingga migrasi dari MySQL ke Cloud NoSQL dapat dilakukan dengan hanya mengganti implementasi tanpa mengubah kode kelas tingkat tinggi. Dengan penerapan SOLID Principles, sistem SIAKAD menjadi tangkas, modular, dan siap mengakomodasi perubahan di masa depan secara elegan.